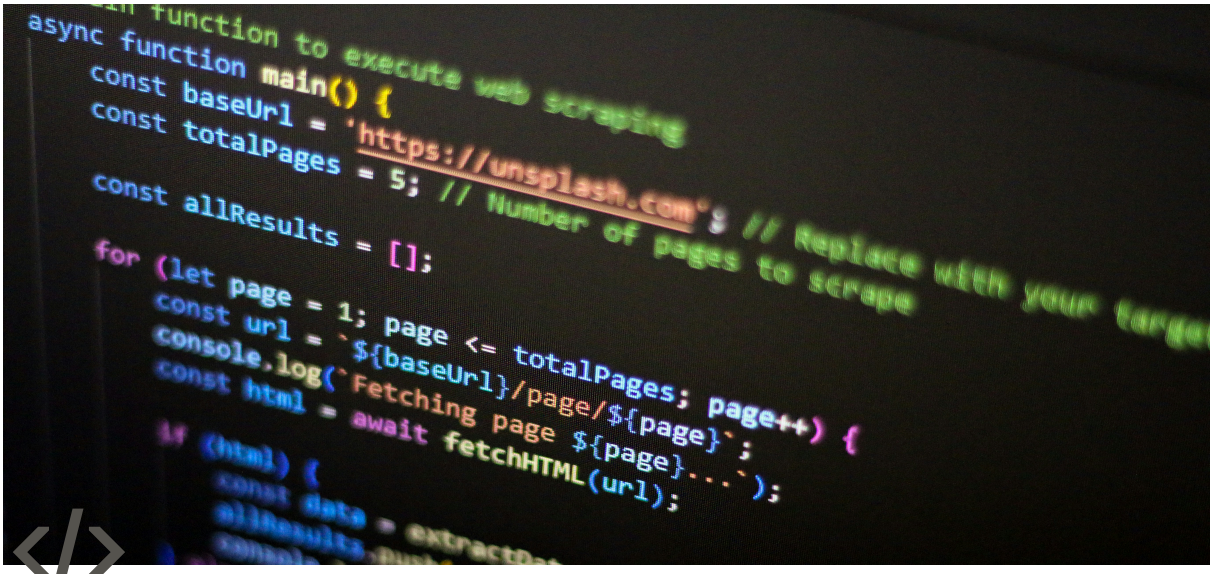

Sistema de Gerenciamento de Lava jato

#

Guia Completo: Construindo seu Sistema Web (Responsivo)

Um roteiro passo a passo para sair da ideia até um sistema funcionando, pensado para quem está começando agora e quer usar em um negócio pequeno ou uso pessoal.
O sistema é **web**, mas construído para funcionar bem tanto no computador quanto no celular (responsivo).

Visão geral das fases

1. **Documentação e planejamento**
2. **Escolha de tecnologia**
3. **Modelagem de dados**
4. **Configuração do ambiente**

5. **Desenvolvimento do back-end (API)**
6. **Desenvolvimento do front-end web responsivo**
7. **Testes**
8. **Deploy (colocar no ar)**
9. **Manutenção**

Fase 1 — Documentação e planejamento

Antes de escrever qualquer código, escreva. Isso evita retrabalho.

1.1 Documento de escopo (1-2 páginas)

Responda por escrito:

- **Problema**: que problema esse sistema resolve?
- **Usuários**: quem vai usar (você, clientes, funcionários)?
- **Funcionalidades essenciais (MVP)**: liste só o mínimo necessário para funcionar. Corte tudo que for "seria legal ter".
- **Fora de escopo**: o que você propositalmente NÃO vai fazer agora.

1.2 Casos de uso

Para cada funcionalidade, escreva uma frase simples:

> "Como [tipo de usuário], eu quero [ação], para [benefício]."

Exemplo: "Como dono da loja, eu quero cadastrar produtos, para controlar meu estoque."

1.3 Wireframes (esboços de tela)

Não precisa ser bonito — pode ser desenho no papel ou uma ferramenta simples (Figma tem plano gratuito).

Como o sistema é responsivo, vale a pena esboçar **duas versões de cada tela principal**: uma larga (desktop) e uma estreita (celular), para já pensar como os elementos vão se reorganizar.

1.4 Requisitos não-funcionais

Pense em:

- Quantos usuários simultâneos você espera (provavelmente poucos no início)
- Em quais tamanhos de tela o sistema precisa funcionar bem (celular, tablet, desktop)
- Dados sensíveis (LGPD) — CPF, pagamentos, etc.

Entregável desta fase: um documento markdown ou PDF com escopo, casos de uso e telas esboçadas (desktop + mobile).

Fase 2 — Escolha de tecnologia (stack)

Para iniciante construindo um sistema web responsivo, a stack mais comum e com boa curva de aprendizado:

Camada	Tecnologia sugerida	Por quê
Back-end/API	Node.js + Express (ou NestJS)	JavaScript também no back, menos contexto novo pra aprender
Banco de dados	PostgreSQL (ou SQLite para começar)	Robusto, gratuito, muito usado no mercado
Front-end web	React (com Vite)	Mais usado no mercado, muita documentação, ótimo suporte a responsividade
Estilização/responsividade	Tailwind CSS	Sistema de classes prontas para adaptar o layout por tamanho de tela, sem escrever CSS do zero
Autenticação	Firebase Auth ou Supabase Auth	Evita programar login do zero (senha, recuperação, etc.)
Hospedagem back-end	Railway ou Render (plano free)	Deploy simples, sem configurar servidor manualmente
Hospedagem front-end	Vercel ou Netlify	Deploy automático a cada mudança

> **Alternativa ainda mais simples para começar**: usar **Supabase** (banco de dados + autenticação + API pronta) e programar só o front-end web consumindo essa API. Isso reduz muito o trabalho de back-end no início.

Entregável: decisão registrada no documento (ex: "vamos usar Node+Express+PostgreSQL+React+Tailwind").

Fase 3 — Modelagem de dados

Antes de codar, desenhe as tabelas do banco:

1. Liste as "coisas" do seu sistema (ex: Usuário, Produto, Pedido)
2. Para cada uma, liste os campos (ex: Produto: nome, preço, quantidade)
3. Desenhe como elas se relacionam (um Pedido tem vários Produtos, etc.)

Ferramenta gratuita: dbdiagram.io — você escreve texto simples e ele gera o diagrama.

Entregável: diagrama do banco de dados (Entidade-Relacionamento).

Fase 4 — Configuração do ambiente

1. Instalar **Node.js** (versão LTS) no computador
2. Instalar um editor de código: **VS Code** (gratuito)
3. Criar uma conta no **GitHub** (versionamento de código — essencial mesmo trabalhando sozinho, serve como backup e histórico)
4. Instalar **Git** e aprender os comandos básicos: `git init`, `git add`, `git commit`, `git push`
5. Criar o repositório do projeto no GitHub

Entregável: ambiente pronto, repositório criado no GitHub.

Fase 5 — Desenvolvimento do back-end (API)

Passo a passo:

1. Iniciar projeto Node.js (`npm init`)
2. Instalar Express (`npm install express`)
3. Criar as rotas básicas seguindo o padrão **CRUD** (Create, Read, Update, Delete) para cada entidade do seu modelo de dados
    - Exemplo para Produto: `POST /produtos`, `GET /produtos`, `GET /produtos/:id`, `PUT /produtos/:id`, `DELETE /produtos/:id`

4. Conectar ao banco de dados (usar um ORM como **Prisma** facilita muito para iniciantes)
5. Implementar autenticação (login/cadastro) — pode usar Supabase/Firebase para não programar do zero
6. Adicionar validação de dados de entrada (ex: biblioteca `zod`)
7. Tratar erros de forma consistente (respostas de erro padronizadas)

Boas práticas desde já:

- Nunca deixar senhas de banco de dados direto no código — usar arquivo `.env`
- Organizar em pastas: `routes/`, `controllers/`, `models/`

Entregável: API funcionando, testável via **Postman** ou **Insomnia**.

Fase 6 — Desenvolvimento do front-end web responsivo

1. Criar projeto React com Vite (`npm create vite@latest`)
2. Instalar e configurar o Tailwind CSS
3. Estruturar páginas conforme os wireframes da Fase 1
4. Criar componentes reutilizáveis (botões, cards, formulários) já pensando em responsividade desde o início — é bem mais fácil que adaptar depois
5. Conectar com a API (usar `fetch` ou biblioteca `axios`)
6. Implementar tela de login e proteção de rotas (usuário não logado não acessa telas internas)
7. Cuidar do estado de carregamento e erros (mostrar "carregando...", mensagens de erro)

Como funciona a responsividade na prática

- Definir um **layout mobile-first**: desenhar e codar primeiro pensando na tela pequena, depois adaptar para telas maiores (é mais simples que o caminho inverso)
- Usar os breakpoints do Tailwind (`sm:`, `md:`, `lg:`) para reorganizar o layout conforme a largura da tela — ex: menu lateral no desktop vira menu

"hambúrguer" no celular

- Testar sempre nas ferramentas de desenvolvedor do navegador (modo de simulação de dispositivo móvel), além de testar no celular de verdade
- Cuidar de detalhes que importam em toque (mobile): botões grandes o suficiente para o dedo, espaçamento entre cliques, sem depender de "hover"

Entregável: site funcional, integrado com a API, funcionando bem em desktop, tablet e celular.

Fase 7 — Testes

Para um sistema pequeno, foque no essencial primeiro:

Testes manuais (obrigatório, mesmo com testes automatizados)

- Criar uma checklist com cada caso de uso da Fase 1 e testar manualmente antes de cada entrega
- Testar cada tela em pelo menos 3 larguras: celular, tablet e desktop

Testes automatizados (recomendado, mas pode vir depois do MVP)

- **Back-end**: testes de API com `Jest` + `Supertest` — testar se cada rota responde como esperado
- **Front-end**: testes de componentes com `Vitest` + `React Testing Library`

Teste com usuário real

- Peça para uma pessoa (não você) usar o sistema sem ajuda, no celular dela, e observe onde ela trava. Isso revela problemas de responsividade e usabilidade que você não percebe por já conhecer o sistema.

Entregável: checklist de testes manuais executada (incluindo os 3 tamanhos de tela) + (opcional) suíte de testes automatizados básica.

Fase 8 — Deploy (colocar no ar)

1. **Back-end**: subir no Railway ou Render, conectar ao banco de dados de produção
2. **Front-end web**: subir no Vercel ou Netlify, conectado ao repositório do GitHub (deploy automático a cada `git push`)
3. Configurar variáveis de ambiente de produção (nunca reaproveitar chaves de teste)
4. Configurar backup do banco de dados
5. (Opcional) Configurar um domínio próprio (ex: `seusite.com.br`) apontando para o Vercel/Netlify

Entregável: sistema acessível publicamente por um link, funcionando em qualquer dispositivo com navegador.

Fase 9 — Manutenção

- Monitorar erros (ferramentas gratuitas como **Sentry** avisam quando algo quebra em produção)
- Manter um backlog simples de melhorias (pode ser até uma planilha)
- Fazer backups periódicos do banco de dados
- Revisar dependências desatualizadas de tempos em tempos

Resumo do caminho mais curto para o MVP

Se você quiser o caminho mais rápido possível como iniciante:

1. Supabase (banco + autenticação + API pronta) → economiza toda a Fase 5
2. React + Vite + Tailwind CSS para o site
3. Vercel para hospedar

Isso permite ter algo funcionando, testável e responsivo em poucas semanas, mesmo sendo iniciante, sem abrir mão de aprender os fundamentos — e sem a complexidade extra de manter um app mobile separado.

Próximos passos sugeridos

Posso ajudar detalhando qualquer uma dessas fases — por exemplo, criar o documento de escopo junto com você, desenhar o modelo de dados do seu

caso específico, ou já começar o código de uma das partes. Basta me contar mais sobre o que o sistema vai fazer.

Guia Documentação 1